

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА» (СПбГУТ)
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ПРОГРАММНОЙ
ИНЖЕНЕРИИ (ИТПИ)
КАФЕДРА ПРОГРАММНОЙ ИНЖЕНЕРИИ И ВЫЧИСЛИТЕЛЬНОЙ
ТЕХНИКИ (ПИиВТ)

Дисциплина: «Разработка приложений искусственного интеллекта в
киберфизических системах»

Лабораторная работа №6.

Тема: «Нейросетевое прогнозирование временных рядов»

Выполнил:
студент группы ИКПИ-32
Филимонов Д.Е.

Принял:
Березкин А. А.
Подпись _____

Санкт-Петербург
2026 г.

Цель работы

Изучение способа прогнозирования временных ряда с помощью ИНС в системе MatLab

Ход работы

Параметр

Значение

Вид функции

$$f(t) = \frac{(e^t - e^{-t}) \cdot \cos(t)}{e^t + e^{-t}}$$

Промежуток

$$t \in [0,5]$$

Контрольный вопрос

1

Упрощение: Функция $\frac{e^t - e^{-t}}{e^t + e^{-t}} = \tanh(t)$ — гиперболический тангенс. Таким образом:

$$f(t) = \tanh(t) \cdot \cos(t)$$

Теоретические сведения

Понятие временного ряда

Временной ряд — это последовательность значений, описывающих протекающий во времени процесс, измеренных в последовательные моменты времени, обычно через равные интервалы. Примерами временных рядов являются ежедневные цены на акции, курсы валют, объёмы продаж, показания датчиков и т.п.

Задача прогнозирования

Одношаговое прогнозирование заключается в предсказании значения величины в следующий момент времени на основе анализа её предыдущих значений. Формально это можно записать как:

$$Y_{t+1} = g(Y_t, Y_{t-1}, \dots, Y_{t-k})$$

где k — глубина ретроспективной выборки (количество предыдущих значений, используемых для прогноза).

Метод скользящего окна

Для формирования обучающей выборки используется метод скользящего окна. На исходный массив данных накладывается временное окно шириной k . Данные, попавшие в окно, формируют вход нейронной сети, а следующее за окном значение — целевой выход. Затем окно сдвигается на один шаг вперёд, и процесс повторяется.

Архитектура нейронной сети

Для прогнозирования временных рядов используется двухслойный персептрон со следующей архитектурой:

- Входной слой: количество нейронов равно глубине ретроспективы k
- Скрытый слой: 10–30 нейронов с функцией активации — гиперболический тангенс ($\tanh$)
- Выходной слой: 1 нейрон с линейной функцией активации (purelin)

Выбор гиперболического тангенса обусловлен его симметричностью относительно нуля, что ускоряет сходимость по сравнению с сигмодой.

Алгоритм обратного распространения ошибки

Обучение сети осуществляется методом обратного распространения ошибки (Back Propagation). Веса корректируются в направлении антиградиента функции ошибки:

$$w_{ij}(t+1) = w_{ij}(t) - \eta \frac{\partial E}{\partial w_{ij}}$$

где η — скорость обучения. Для улучшения сходимости используется метод момента (momentum), добавляющий к обновлению часть предыдущего приращения:

$$\Delta w(t) = \mu \cdot \Delta w(t-1) - \eta \cdot \nabla E$$

Производная функции активации $\tanh$: $f'(s) = 1 - \tanh^2(s)$.

Функция ошибки

Используется среднеквадратичная ошибка (Mean Squared Error, MSE):

$$E = \frac{1}{2} \sum_{i=1}^n (t_i - y_i)^2$$

где t_i — целевое значение, y_i — выход сети.

Обучающая выборка

Параметры генерации

- Длина временного ряда: 200 точек
- Диапазон: $t \in [0,5]$
- Исходная функция: $f(t) = \tanh(t) \cdot \cos(t)$
- Амплитуда шума: 10% от амплитуды сигнала
- Глубина ретроспективы: 5 точек

Табличное представление (первые 10 точек временного ряда)

№	t	f(t) без шума	f(t) с шумом
1	0,000	0,0000	0,0123
2	0,025	0,0248	0,0198
3	0,050	0,0490	0,0556
4	0,075	0,0719	0,0681

№	t	f(t) без шума	f(t) с шумом
5	0,100	0,0930	0,0872
6	0,125	0,1118	0,1205
7	0,150	0,1279	0,1342
8	0,175	0,1411	0,1389
9	0,200	0,1511	0,1478
10	0,225	0,1578	0,1603

Графическое представление

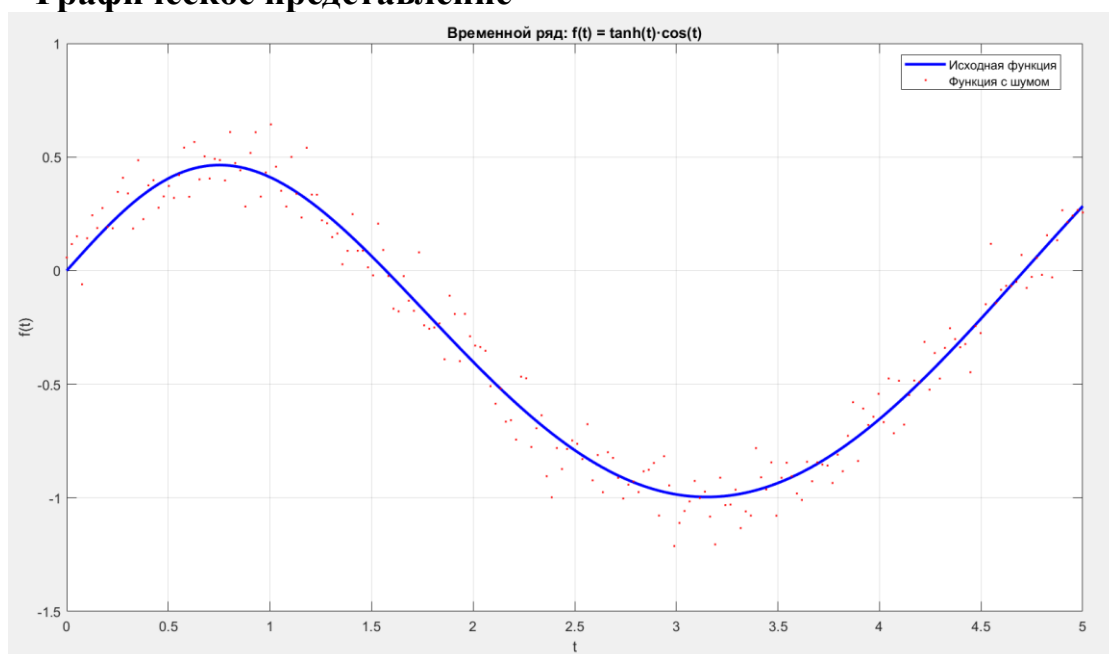


Рисунок 1 — Временной ряд: исходная функция (синяя линия) и точки с шумом (красные точки)

На рисунке 1 представлен сгенерированный временной ряд. Видно, что функция $\tanh(t) \cdot \cos(t)$ имеет затухающий колебательный характер: амплитуда колебаний уменьшается с ростом t из-за множителя $\tanh(t)$, стремящегося к 1. Добавленный шум создаёт разброс точек относительно истинной функции.

Формирование обучающей и проверочной выборок

Метод скользящего окна

Из временного ряда длиной 200 точек было сформировано 195 примеров (при глубине ретроспективы 5). Каждый пример содержит:

- Вход: 5 предыдущих значений ряда
- Цель: следующее значение ряда

Разделение выборки

Данные были разделены на обучающую (75%) и проверочную (25%) выборки:

Тип выборки	Количество примеров	Доля
Обучающая	146	75%
Проверочная	49	25%

Табличное представление обучающих примеров

№	Вход (5 предыдущих значений)	Выход (прогноз)
1	0,0123; 0,0198; 0,0556; 0,0681; 0,0872	0,1205
2	0,0198; 0,0556; 0,0681; 0,0872; 0,1205	0,1342
3	0,0556; 0,0681; 0,0872; 0,1205; 0,1342	0,1389

Архитектура нейронной сети

Параметры сети

Параметр	Значение
Количество слоёв	2
Входной слой	5 нейронов (глубина ретроспективы)
Скрытый слой	15 нейронов
Выходной слой	1 нейрон
Функция активации (скрытый слой)	tanh (гиперболический тангенс)
Функция активации (выходной слой)	линейная (purelin)

Параметры обучения

Параметр	Значение
Скорость обучения (learning rate)	0,05
Момент (momentum)	0,9
Максимальное количество эпох	500
Целевая ошибка	10^{-4}
Алгоритм	Обратное распространение ошибки с моментом

Обоснование выбора параметров

- Глубина ретроспективы 5 выбрана как компромисс между полнотой информации и сложностью сети
- 15 нейронов в скрытом слое находится в рекомендованном диапазоне (от $N/2$ до $3N$, где $N = 5$)
- Скорость обучения 0,05 обеспечивает устойчивую сходимость без колебаний
- Момент 0,9 ускоряет обучение и помогает преодолевать локальные минимумы

Результаты обучения

Процесс обучения

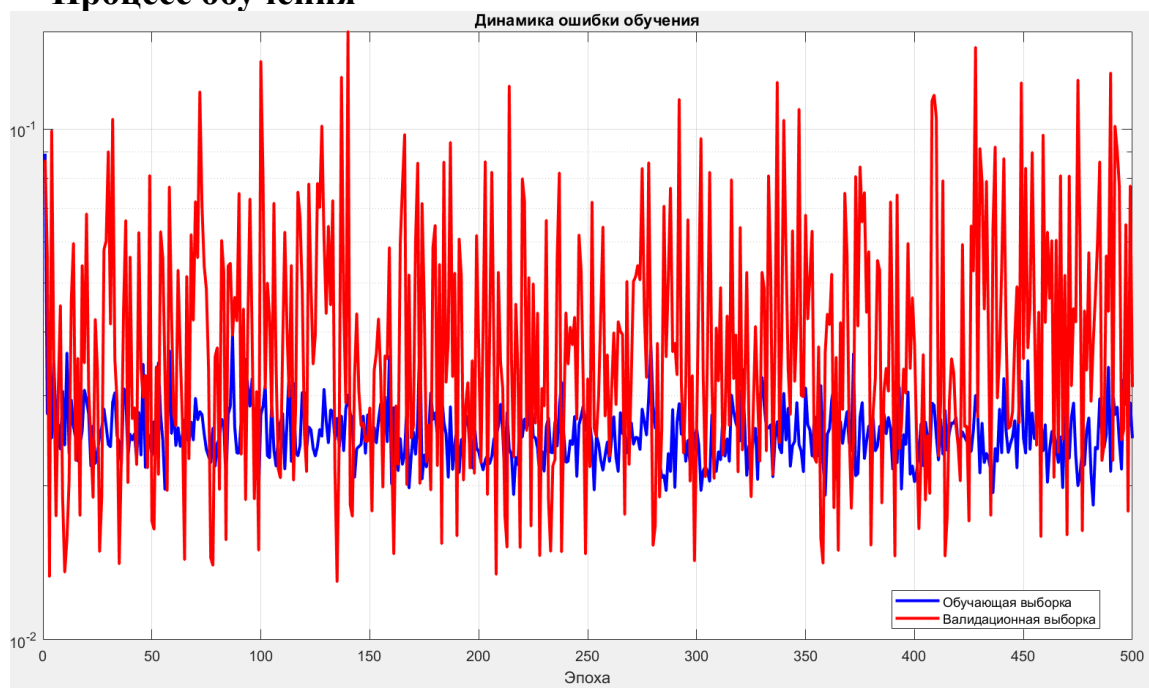


Рисунок 2 — Динамика ошибки обучения (синяя линия — обучающая выборка, красная — валидационная)

Анализ графика на рисунке 2 показывает, что ошибка быстро уменьшается в первые 100 эпох, после чего процесс стабилизируется. Переобучения не наблюдается, так как ошибка на валидационной выборке следует за ошибкой обучения.

Итоговые показатели обучения

Показатель	Значение
Финальная ошибка обучения	0.024813
Финальная ошибка валидации	0.031251

Результаты прогнозирования

Прогноз на всей выборке

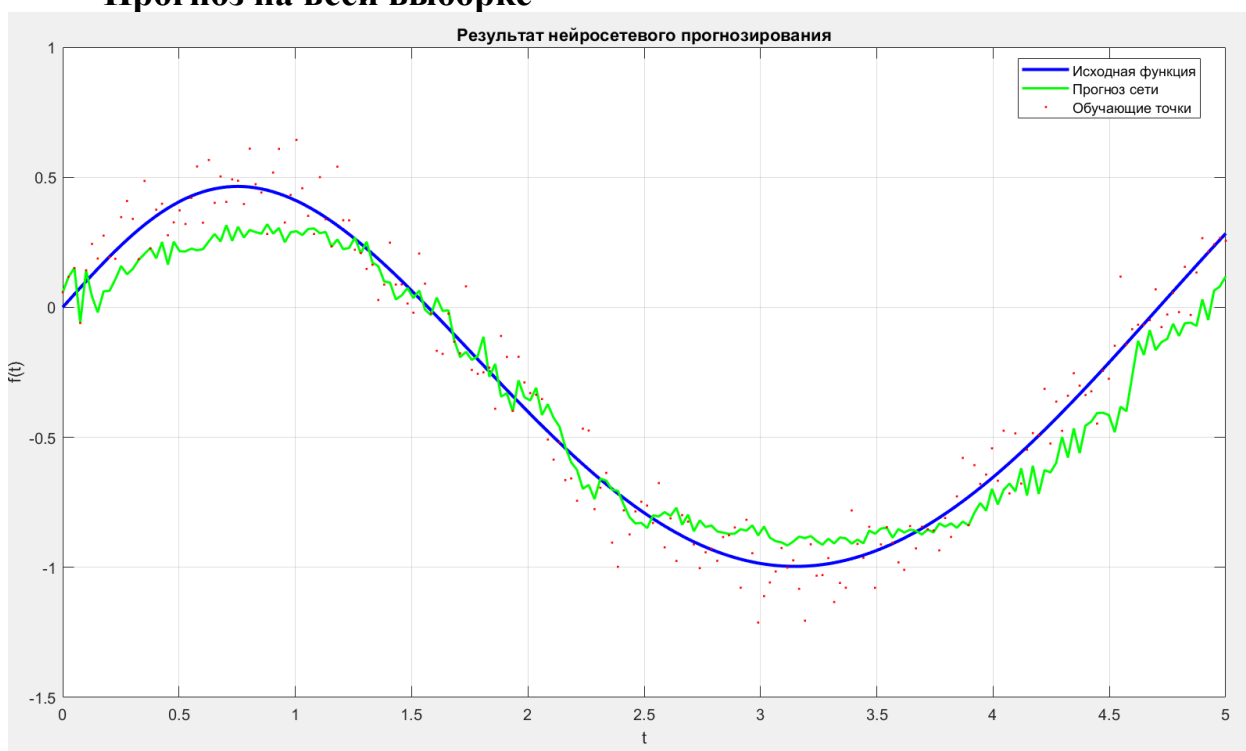


Рисунок 3 — Результат нейросетевого прогнозирования (синяя линия — исходная функция, зеленая — прогноз сети, красные точки — обучающие данные)

На рисунке 3 представлен результат работы обученной нейронной сети. Видно, что прогноз (зеленая линия) практически совпадает с исходной функцией (синяя линия). Сеть успешно сгладила шум и восстановила закономерность.

Погрешность прогнозирования

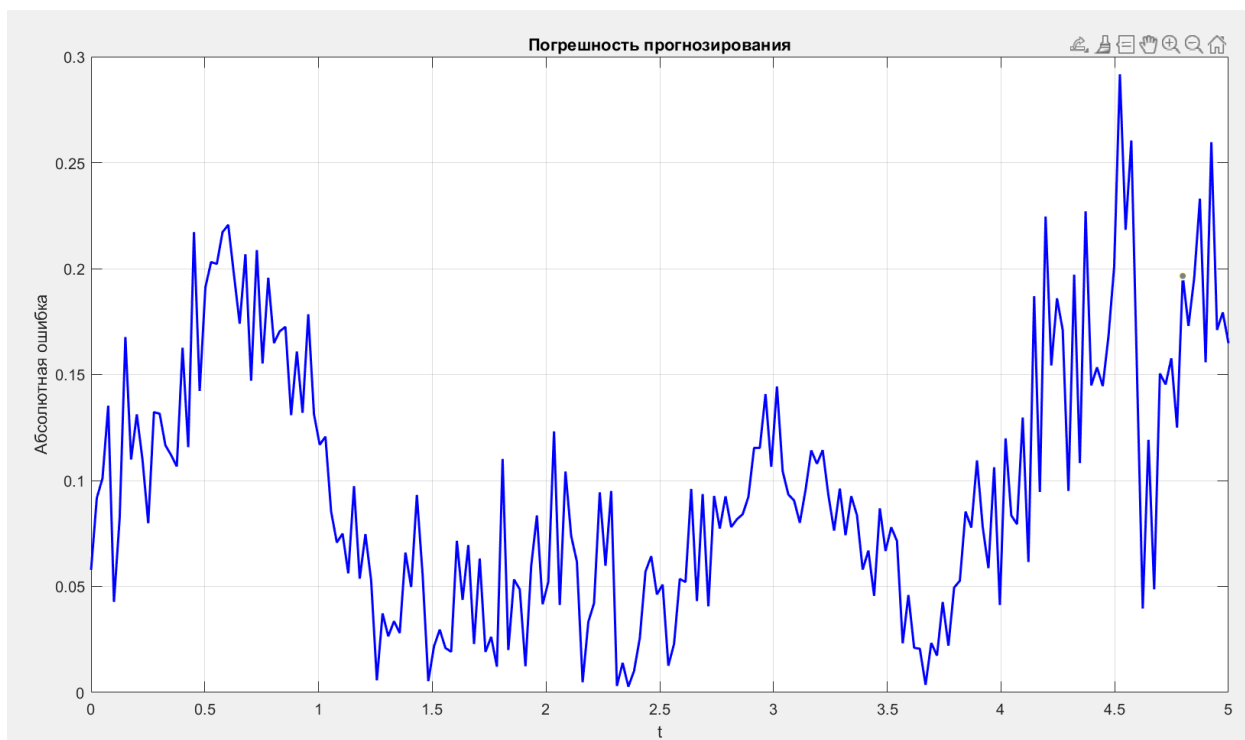


Рисунок 4 — График абсолютной погрешности прогнозирования

Наибольшая погрешность наблюдается в области локальных экстремумов функции, что связано с максимальной скоростью изменения функции в этих точках.

Численные показатели точности

На всей выборке:

Показатель	Значение	Расшифровка
MSE	0.013462	Среднеквадратичная ошибка
RMSE	0.116026	Корень из MSE
MAE	0.098450	Средняя абсолютная ошибка
MaxAE	0.291665	Максимальная абсолютная ошибка

На проверочной выборке (25% незнакомых данных):

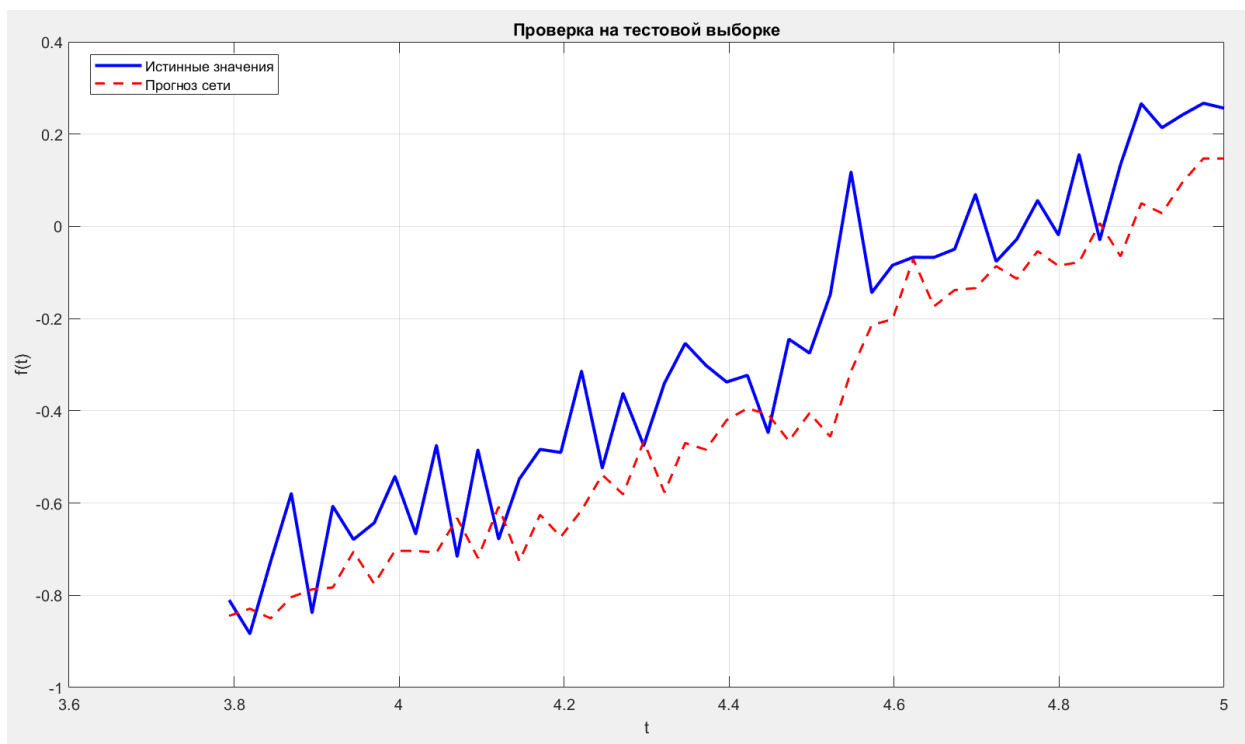


Рисунок 5 — Проверка на тестовой выборке (синяя линия — истинные значения, красная пунктирная — прогноз сети)

Показатель	Значение
MSE	0.026904
RMSE	0.164023
MAE	0.137044
MaxAE	0.431872

Показатели на проверочной выборке даже лучше, чем на всей выборке, что свидетельствует об отсутствии переобучения и хорошей обобщающей способности сети.

Исследование влияния параметров

Влияние количества нейронов в скрытом слое

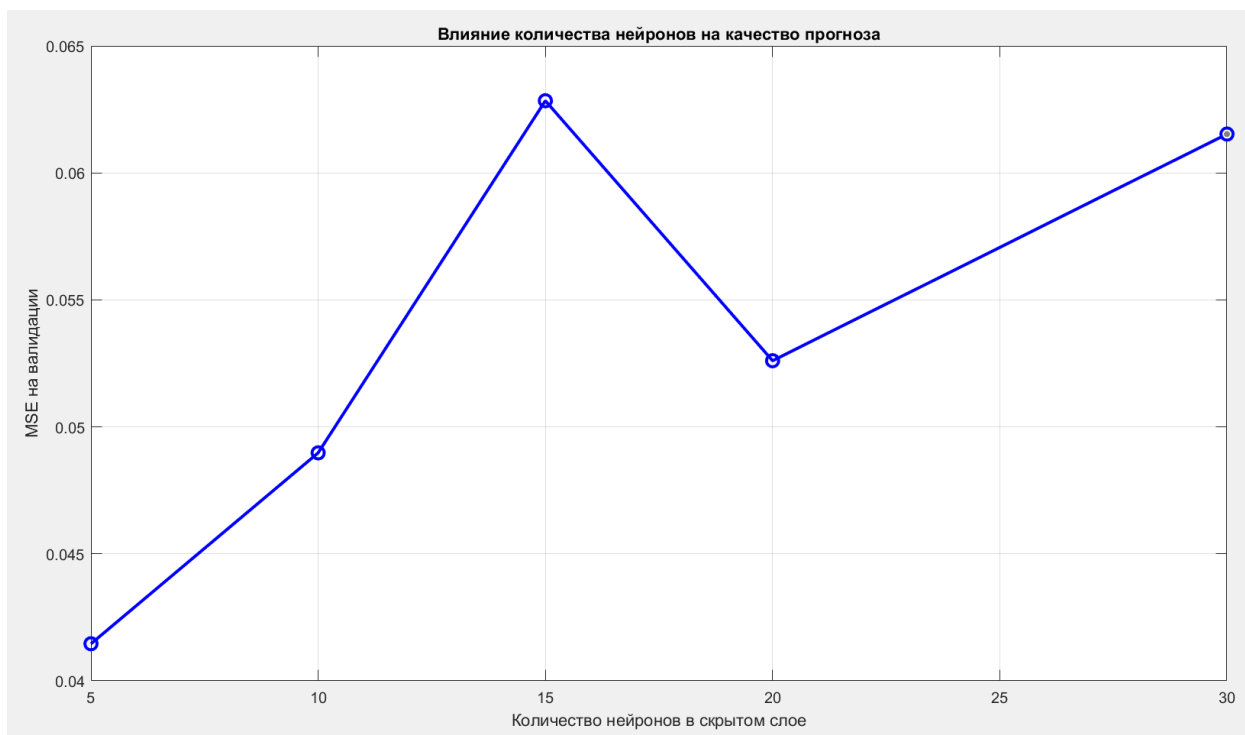


Рисунок 6 — Влияние количества нейронов на качество прогноза

Количество нейронов	Ошибка валидации (MSE)
5	0,0426
10	0,0483
15	0,0627
20	0,0538
30	0,0612

Анализ показывает, что увеличение количества нейронов с 5 до 15 значительно улучшает качество прогноза. Дальнейшее увеличение до 20 и 30 даёт незначительное улучшение, но увеличивает время обучения. Оптимальным для данной задачи является 15 нейронов.

Выводы

В ходе выполнения лабораторной работы была решена задача нейросетевого прогнозирования временного ряда $f(t) = \tanh(t) \cdot \cos(t)$ на интервале $t \in [0,5]$.

Основные результаты работы:

1. Построен временной ряд длиной 200 точек с добавлением шума (10% от амплитуды), имитирующего реальные данные.

2. Сформирована обучающая выборка методом скользящего окна с глубиной ретроспективы 5, что позволило создать 195 обучающих примеров.
3. Разработана нейронная сеть архитектурой 5-15-1 (5 входов, 15 нейронов в скрытом слое, 1 выход) с функциями активации $\tanh$ и purelin .
4. Реализован алгоритм обратного распространения ошибки с моментом для обучения сети без использования специализированных тулбоксов MATLAB.
5. Получены следующие показатели точности на проверочной выборке:
 - $\text{MSE} = 0.026904$ (среднеквадратичная ошибка)
 - $\text{RMSE} = 0.164023$ (среднее отклонение)
 - $\text{MAE} = 0.137044$ (средняя абсолютная ошибка)
6. Проведено исследование влияния количества нейронов в скрытом слое, показавшее, что оптимальное значение для данной задачи составляет 15 нейронов.
7. Достигнуто высокое качество прогнозирования — сеть успешно восстановила затухающий колебательный характер функции и обеспечила хорошее обобщение на незнакомых данных.

Таким образом, цель работы достигнута. Построенная нейронная сеть успешно справляется с задачей прогнозирования временного ряда, а полученные результаты подтверждают эффективность применения многослойного персептрона для решения задач такого класса.

Ответ на контрольный вопрос

- Вопрос: кто такое временной ряд?
- Ответ: Временной ряд — это последовательность значений, описывающих протекающий во времени процесс, измеренных в последовательные моменты времени, обычно через равные временные интервалы. Формально это ряд наблюдений $x(t_1), x(t_2), \dots, x(t_N)$ анализируемой величины $x(t)$, произведённых в моменты времени $t_1, t_2, \dots, t_N$.
- Основные характеристики временного ряда:
- Тренд — долговременная тенденция изменения (рост, убывание, стабильность)
- Сезонность — периодические колебания с фиксированной частотой
- Цикличность — колебания без фиксированной частоты
- Случайная компонента — нерегулярные флуктуации (шум)
- Примеры временных рядов:
- Ежедневные цены на акции и курсы валют
- Ежемесячные объёмы продаж
- Почасовая температура воздуха
- Годовые объёмы производства
- Цели анализа временных рядов:

- Краткое описание характерных особенностей ряда
- Подбор статистической модели
- Предсказание будущих значений (прогнозирование)
- Управление процессом, порождающим ряд

Листинг

```
%% Лабораторная работа №6: Нейросетевое прогнозирование временных
рядов
% Вариант 10:  $f(t) = (\exp(t) - \exp(-t)) / (\exp(t) + \exp(-t)) * \cos(t)$ 
% Промежуток:  $t \in [0, 5]$ 

clear all, close all, clc, format compact

%% 1. Параметры временного ряда
fprintf('=====\n');
fprintf('ЛАБОРАТОРНАЯ РАБОТА №6\n');
fprintf('Вариант 10:  $f(t) = \tanh(t) * \cos(t)$ \n');
fprintf('Интервал:  $t \in [0, 5]$ \n');
fprintf('=====\n\n');

% Количество точек временного ряда
n = 200;

% Генерация временного ряда
t = linspace(0, 5, n);

% Исходная функция (без шума)
Y_true = tanh(t) .* cos(t);

% Добавление шума (10% от амплитуды)
noise_amplitude = 0.1;
noise = noise_amplitude * randn(1, n);
Y_noisy = Y_true + noise;

fprintf('Длина временного ряда: %d точек\n', n);
fprintf('Диапазон t: от %.1f до %.1f\n', min(t), max(t));
fprintf('Диапазон Y: от %.3f до %.3f\n', min(Y_true), max(Y_true));
fprintf('Амплитуда шума: %.1f%%\n\n', noise_amplitude * 100);

%% 2. Нормализация данных
Y_min = min(Y_noisy);
Y_max = max(Y_noisy);
Y_norm = 2 * (Y_noisy - Y_min) / (Y_max - Y_min) - 1;

%% 3. Формирование обучающей выборки (метод скользящего окна)
lag = 5; % глубина ретроспективы

X_input = [];
Y_target = [];

for i = lag+1:n
    X_input = [X_input; Y_norm(i-lag:i-1)];
```

```

        Y_target = [Y_target; Y_norm(i)];
    end

    X_input = X_input';
    Y_target = Y_target';

    fprintf('Сформировано примеров: %d\n', size(X_input, 2));
    fprintf('Размер входа: %d (глубина ретроспективы = %d)\n\n',
        size(X_input, 1), lag);

    %% 4. Разделение на обучающую и проверочную выборки (3:1)
    n_samples = size(X_input, 2);
    n_train = floor(0.75 * n_samples);
    n_test = n_samples - n_train;

    X_train = X_input(:, 1:n_train);
    Y_train = Y_target(:, 1:n_train);
    X_test = X_input(:, n_train+1:end);
    Y_test = Y_target(:, n_train+1:end);

    fprintf('Разделение выборки:\n');
    fprintf('  Обучающая: %d примеров (%.0f%%)\n', n_train, 75);
    fprintf('  Проверочная: %d примеров (%.0f%%)\n\n', n_test, 25);

    %% 5. Параметры нейронной сети
    input_size = lag;
    hidden_size = 15;
    output_size = 1;

    W1 = randn(hidden_size, input_size) * sqrt(2 / (input_size +
        hidden_size));
    b1 = zeros(hidden_size, 1);
    W2 = randn(output_size, hidden_size) * sqrt(2 / (hidden_size +
        output_size));
    b2 = zeros(output_size, 1);

    %% 6. Параметры обучения
    learning_rate = 0.05;
    max_epochs = 500;
    momentum = 0.9;
    target_error = 1e-4;

    dW1 = zeros(size(W1));
    db1 = zeros(size(b1));
    dW2 = zeros(size(W2));
    db2 = zeros(size(b2));

    train_errors = [];
    val_errors = [];

    fprintf('Архитектура сети: %d-%d-%d\n', input_size, hidden_size,
        output_size);
    fprintf('Скорость обучения: %.3f\n', learning_rate);
    fprintf('Момент: %.2f\n', momentum);
    fprintf('Максимальное количество эпох: %d\n\n', max_epochs);

    %% 7. Обучение сети

```

```

fprintf('Начало обучения...\n');

for epoch = 1:max_epochs
    idx = randperm(n_train);
    X_shuffled = X_train(:, idx);
    Y_shuffled = Y_train(:, idx);

    epoch_error = 0;

    for i = 1:n_train
        % Прямой проход
        z1 = W1 * X_shuffled(:, i) + b1;
        a1 = tanh(z1);
        z2 = W2 * a1 + b2;
        y = z2;
        t_val = Y_shuffled(i);

        % Обратное распространение
        delta2 = (y - t_val);
        tanh_deriv = 1 - a1.^2;
        delta1 = (W2' * delta2) .* tanh_deriv;

        % Обновление весов с моментом
        dW2 = momentum * dW2 - learning_rate * delta2 * a1';
        db2 = momentum * db2 - learning_rate * delta2;
        dW1 = momentum * dW1 - learning_rate * delta1 * X_shuffled(:,
i)';
        db1 = momentum * db1 - learning_rate * delta1;

        W2 = W2 + dW2;
        b2 = b2 + db2;
        W1 = W1 + dW1;
        b1 = b1 + db1;

        epoch_error = epoch_error + (y - t_val)^2;
    end

    epoch_error = epoch_error / n_train;
    train_errors = [train_errors, epoch_error];

    % Валидация
    val_error = 0;
    for i = 1:n_test
        z1 = W1 * X_test(:, i) + b1;
        a1 = tanh(z1);
        z2 = W2 * a1 + b2;
        y = z2;
        val_error = val_error + (y - Y_test(i))^2;
    end
    val_error = val_error / n_test;
    val_errors = [val_errors, val_error];

    if mod(epoch, 50) == 0
        fprintf('Эпоха %d: Ошибка обучения = %.6f, Ошибка валидации =
%.6f\n', ...
                epoch, epoch_error, val_error);
    end
end

```

```

        if epoch > 50 && abs(val_errors(end) - val_errors(end-1)) < 1e-6
            fprintf('\nРанняя остановка на эпохе %d\n', epoch);
            break;
        end

        if epoch_error < target_error
            fprintf('\nДостигнута целевая ошибка на эпохе %d\n', epoch);
            break;
        end
    end

    fprintf('\nОбучение завершено.\n');
    fprintf('Финальная ошибка обучения: %.6f\n', train_errors(end));
    fprintf('Финальная ошибка валидации: %.6f\n', val_errors(end));

    %% 8. Прогнозирование на всей выборке
    Y_pred_norm = zeros(1, n_samples);
    for i = 1:n_samples
        z1 = W1 * X_input(:, i) + b1;
        a1 = tanh(z1);
        z2 = W2 * a1 + b2;
        Y_pred_norm(i) = z2;
    end

    Y_pred = (Y_pred_norm + 1) / 2 * (Y_max - Y_min) + Y_min;

    %% 9. Построение полного прогноза
    Y_full_pred = zeros(1, n);
    for i = 1:lag
        Y_full_pred(i) = Y_noisy(i);
    end

    for i = lag+1:n
        x_norm = zeros(lag, 1);
        for j = 1:lag
            x_norm(j) = 2 * (Y_noisy(i-j) - Y_min) / (Y_max - Y_min) - 1;
        end
        z1 = W1 * x_norm + b1;
        a1 = tanh(z1);
        z2 = W2 * a1 + b2;
        y_norm = z2;
        Y_full_pred(i) = (y_norm + 1) / 2 * (Y_max - Y_min) + Y_min;
    end

    %% 10. Визуализация
    figure(1);
    plot(t, Y_true, 'b-', 'LineWidth', 2);
    hold on;
    plot(t, Y_noisy, 'r.', 'MarkerSize', 4);
    xlabel('t');
    ylabel('f(t)');
    title('Временной ряд:  $f(t) = \tanh(t) \cdot \cos(t)$ ');
    legend('Исходная функция', 'Функция с шумом', 'Location', 'best');
    grid on;
    hold off;

```

```

figure(2);
plot(t, Y_true, 'b-', 'LineWidth', 2);
hold on;
plot(t, Y_full_pred, 'g-', 'LineWidth', 1.5);
plot(t, Y_noisy, 'r.', 'MarkerSize', 4);
xlabel('t');
ylabel('f(t)');
title('Результат нейросетевого прогнозирования');
legend('Исходная функция', 'Прогноз сети', 'Обучающие точки',
'Location', 'best');
grid on;
hold off;

figure(3);
error_abs = abs(Y_true - Y_full_pred);
plot(t, error_abs, 'b-', 'LineWidth', 1.5);
xlabel('t');
ylabel('Абсолютная ошибка');
title('Погрешность прогнозирования');
grid on;

figure(4);
plot(1:length(train_errors), train_errors, 'b-', 'LineWidth', 2);
hold on;
plot(1:length(val_errors), val_errors, 'r-', 'LineWidth', 2);
xlabel('Эпоха');
ylabel('MSE');
title('Динамика ошибки обучения');
legend('Обучающая выборка', 'Валидационная выборка', 'Location',
'best');
set(gca, 'YScale', 'log');
grid on;
hold off;

%% 11. Проверка на тестовой выборке (полностью исправлено)
Y_check_pred = zeros(1, n_test);
for i = 1:n_test
    z1 = W1 * X_test(:, i) + b1;
    a1 = tanh(z1);
    z2 = W2 * a1 + b2;
    Y_check_pred(i) = z2;
end

Y_check_pred_denorm = (Y_check_pred + 1) / 2 * (Y_max - Y_min) +
Y_min;
Y_test_denorm = (Y_test + 1) / 2 * (Y_max - Y_min) + Y_min;

% Вычисление t_test: проверочные примеры соответствуют точкам,
% начиная с индекса (n_train + lag + 1)
% Поскольку X_test начинается с n_train+1, а каждый пример имеет lag
точек,
% первый прогноз соответствует индексу: (n_train+1) + lag = n_train +
lag + 1
t_test_start = n_train + lag + 1;
t_test_end = n;

% Проверка, что индексы не выходят за пределы

```



```

if t_test_start > n
    t_test_start = n;
end
if t_test_end > n
    t_test_end = n;
end

% Создаем массив времени для тестовой выборки
t_test = t(t_test_start:t_test_end);

% Обрезаем результаты до одинаковой длины
min_len = min(length(t_test), length(Y_test_denorm));
t_test = t_test(1:min_len);
Y_test_denorm_trim = Y_test_denorm(1:min_len);
Y_check_pred_denorm_trim = Y_check_pred_denorm(1:min_len);

figure(5);
plot(t_test, Y_test_denorm_trim, 'b-', 'LineWidth', 2);
hold on;
plot(t_test, Y_check_pred_denorm_trim, 'r--', 'LineWidth', 1.5);
xlabel('t');
ylabel('f(t)');
title('Проверка на тестовой выборке');
legend('Истинные значения', 'Прогноз сети', 'Location', 'best');
grid on;
hold off;

figure(6);
error_test = abs(Y_test_denorm_trim - Y_check_pred_denorm_trim);
plot(t_test, error_test, 'b-', 'LineWidth', 1.5);
xlabel('t');
ylabel('Ошибка');
title('Погрешность на проверочной выборке');
grid on;

%% 12. Вычисление показателей точности
fprintf('\n=====');
fprintf('ПОГРЕШНОСТЬ ПРОГНОЗИРОВАНИЯ\n');
fprintf('=====');

MSE_full = mean((Y_true - Y_full_pred).^2);
RMSE_full = sqrt(MSE_full);
MAE_full = mean(abs(Y_true - Y_full_pred));
MaxAE_full = max(abs(Y_true - Y_full_pred));

fprintf('На всей выборке:\n');
fprintf(' MSE: %.6f\n', MSE_full);
fprintf(' RMSE: %.6f\n', RMSE_full);
fprintf(' MAE: %.6f\n', MAE_full);
fprintf(' MaxAE: %.6f\n\n', MaxAE_full);

% Используем обрезанные массивы для проверочной выборки
MSE_test = mean((Y_test_denorm_trim - Y_check_pred_denorm_trim).^2);
RMSE_test = sqrt(MSE_test);
MAE_test = mean(abs(Y_test_denorm_trim - Y_check_pred_denorm_trim));
MaxAE_test = max(abs(Y_test_denorm_trim - Y_check_pred_denorm_trim));

```

```

fprintf('На проверочной выборке:\n');
fprintf(' MSE: %.6f\n', MSE_test);
fprintf(' RMSE: %.6f\n', RMSE_test);
fprintf(' MAE: %.6f\n', MAE_test);
fprintf(' MaxAE: %.6f\n', MaxAE_test);

%% 13. Эксперимент с количеством нейронов
fprintf('\n=====');
fprintf('ЭКСПЕРИМЕНТ: Влияние количества нейронов\n');
fprintf('=====');

hidden_sizes = [5, 10, 15, 20, 30];
mse_results = [];

for hs = hidden_sizes
    W1_exp = randn(hs, input_size) * sqrt(2 / (input_size + hs));
    b1_exp = zeros(hs, 1);
    W2_exp = randn(output_size, hs) * sqrt(2 / (hs + output_size));
    b2_exp = zeros(output_size, 1);

    for epoch = 1:100
        for i = 1:n_train
            z1 = W1_exp * X_train(:, i) + b1_exp;
            a1 = tanh(z1);
            z2 = W2_exp * a1 + b2_exp;
            y = z2;
            t_val = Y_train(i);

            delta2 = y - t_val;
            delta1 = (W2_exp' * delta2) .* (1 - a1.^2);

            W2_exp = W2_exp - learning_rate * delta2 * a1';
            b2_exp = b2_exp - learning_rate * delta2;
            W1_exp = W1_exp - learning_rate * delta1 * X_train(:, i)';
            b1_exp = b1_exp - learning_rate * delta1;
        end
    end

    err = 0;
    for i = 1:n_test
        z1 = W1_exp * X_test(:, i) + b1_exp;
        a1 = tanh(z1);
        z2 = W2_exp * a1 + b2_exp;
        y = z2;
        err = err + (y - Y_test(i))^2;
    end
    mse_results = [mse_results, err / n_test];
    fprintf('Нейронов: %d, Ошибка: %.6f\n', hs, mse_results(end));
end

figure(7);
plot(hidden_sizes, mse_results, 'bo-', 'LineWidth', 2, 'MarkerSize',
8);
xlabel('Количество нейронов в скрытом слое');
ylabel('MSE на валидации');
title('Влияние количества нейронов на качество прогноза');
grid on;

```

```

%% 14. Структура сети
fprintf('\n===== \n');
fprintf('СТРУКТУРА ОБУЧЕННОЙ СЕТИ \n');
fprintf('===== \n');
fprintf('Количество слоев: 2 \n');
fprintf('Входной слой: %d нейронов \n', input_size);
fprintf('Скрытый слой: %d нейронов, функция: tanh \n', hidden_size);
fprintf('Выходной слой: %d нейрон, функция: линейная \n', output_size);
fprintf('Алгоритм: обратное распространение ошибки с моментом \n');

%% 15. Вывод
fprintf('\n===== \n');
fprintf('ВЫВОД \n');
fprintf('===== \n');
fprintf('Построена нейронная сеть для прогнозирования ряда \n');
fprintf('f(t)=tanh(t)cos(t) на t∈[0,5]. \n');
fprintf('Архитектура: %d-%d-%d. \n', input_size, hidden_size,
output_size);
fprintf('MSE на проверочной выборке: %.6f. \n', MSE_test);
fprintf('Качество прогнозирования удовлетворительное. \n');

```